

Laravel 5: What's New?



Laravel is a free, open source, PHP Web application framework. Laravel v5 was launched in February 2015. This article is about how different it is from the older versions.

Earlier this year, PHP developers were blessed with Laravel 5, the new version of the popular PHP framework, which is a major release with some great, exciting features and alterations in the framework architecture. This new version comes with dozens of fruitful alterations and it's tough to keep track of all of them. Hence, I'll outline a few new features in this article just to let developers know about them.

New directory structure

The major change is in the directory structure. The app folder is broken down into various folders, which include Commands, Console, Events, Exceptions, Handlers, Http, Providers and Services. The folders which moved out from the app folder are Config, Database, Resources and Storage. Classes are namespaced in Laravel 5 and the default namespace is *app*. This can be changed by using the `php artisan app:name <NEW APP NAME>` command. It'll be reflected in all the files, so you don't need to worry about any manual modification. The *config* folder contains various configuration files for Apps, Authentication, Databases, Mail, Sessions, and so on. Application's assets, language and view files are wrapped inside the corresponding sub-folders under the Resources folder. The Bootstrap, Vendor and Tests directories retain the same positions as in the older version.

Method injection

Besides constructor injection, we may also type-hint (specify the expected argument data type in the function declaration) dependency on our controller's methods. The following code shows how method injection works in Laravel 5:

Test.php	MyController.php
<pre><?php namespace App; class test { public function my_function() { return 'Message'; } }</pre>	<pre><?php namespace App\Http\Controller use App\my_function class MyController extends Controller { protected \$test; public function __construct(Test \$test) { \$this->test = \$test; } public function index() { return \$this->test- >my_function(); } }</pre>